Sistemas de Tiempo Real
Departamento de Ingeniería Electrónica
Universidad de Antioquia
2026-2

Práctica No. 4

Controlador Longitudinal de una Aeronave mediante Lingua Franca

Realización: En parejas.

Valor: 10% del curso.

Fecha de entrega: Viernes 23 de octubre del 2026.

Objetivos:

- Modelar un sistema didáctico que regule solamente la velocidad aerodinámica longitudinal de una aeronave simulada.
- Implementar módulos reactivos periódicos mediante **timers**, **reactions**, **ports** y conexiones en **Lingua Franca** (**LF**)[1].
- Explicar cómo los eventos, las *tags* lógicas (tiempo y *microstep*) y las dependencias causales determinan el orden válido de ejecución.
- Demostrar experimentalmente la periodicidad solicitada y evaluar la respuesta del controlador ante cambios de referencia y perturbaciones de medición.

Preguntas orientadoras:

1. ¿Qué diferencia existe entre **reactor**, **reaction**, **timer**, puerto y conexión en **LF**?
2. ¿Qué representa una *tag* (tiempo lógico y *microstep*)? ¿Pueden ejecutarse dos **reactions** con la misma etiqueta temporal? **Explique**.
3. ¿Por qué el período lógico no equivale necesariamente al intervalo físico medido entre el inicio de dos ejecuciones de un módulo o tarea periódica?
4. ¿Qué ventajas ofrece expresar la comunicación mediante puertos y conexiones, frente a implementar manualmente hilos y variables compartidos, usando **POSIX**?

[1] https://www.lf-lang.org/

1. Contexto y Alcance

Esta práctica de laboratorio toma como referencia conceptual el proyecto **Rosace** presentado en el Tutorial sobre **Lingua Franca** realizado recientemente en el **CPS-IoT Week 2026:**

https://github.com/gpatigno/RTS_2026-2/tree/main/MODULO2/LINGUAFRANCA/Tutorial#tutorial-on-lingua-franca-in-cps-iot-week-2026

En dicho proyecto **Rosace** se modelan diferentes funciones de control de vuelo implementadas mediante **reactors** de **Lingua Franca** definidos con distintas frecuencias. En la presente práctica de laboratorio se construirá una versión didáctica y reducida con un único lazo de control, consistente en mantener la velocidad aerodinámica longitudinal cerca de una velocidad de referencia regulando un comando de empuje.

Con ello, en esta práctica no se implementará el control de altitud ni de velocidad vertical definidos originalmente en el proyecto **Rosace**. Así, el modelo a ser implementado **no constituye** un controlador aeronáutico real.

2. Escenario de Control

La aeronave se representa mediante una planta longitudinal discreta, tal como se muestra en la **Figura 1**. El sensor publica una medición de velocidad; el controlador compara esa medición con la referencia y calcula un comando normalizado de empuje; la planta actualiza la velocidad simulada; y el monitor registra los datos para evaluar la periodicidad y la calidad del control.

Variables principales:

- $\boldsymbol{v[k]}(\boldsymbol{m/s})$: velocidad aerodinámica longitudinal simulada.
- $\boldsymbol{v_{ref}[k]}(\boldsymbol{m/s})$: velocidad de referencia.
- $\boldsymbol{v_{med}[k]}(\boldsymbol{m/s})$: velocidad medida por el sensor simulado.
- $\boldsymbol{e_v[k]}(\boldsymbol{m/s})$: error entre referencia y medición.
- $\boldsymbol{u_T[k]}$ (adimensional): comando de empuje normalizado, limitado a [0, 1].
- $\boldsymbol{d_v[k]}(\boldsymbol{m/s})$: perturbación artificial introducida en la medición.

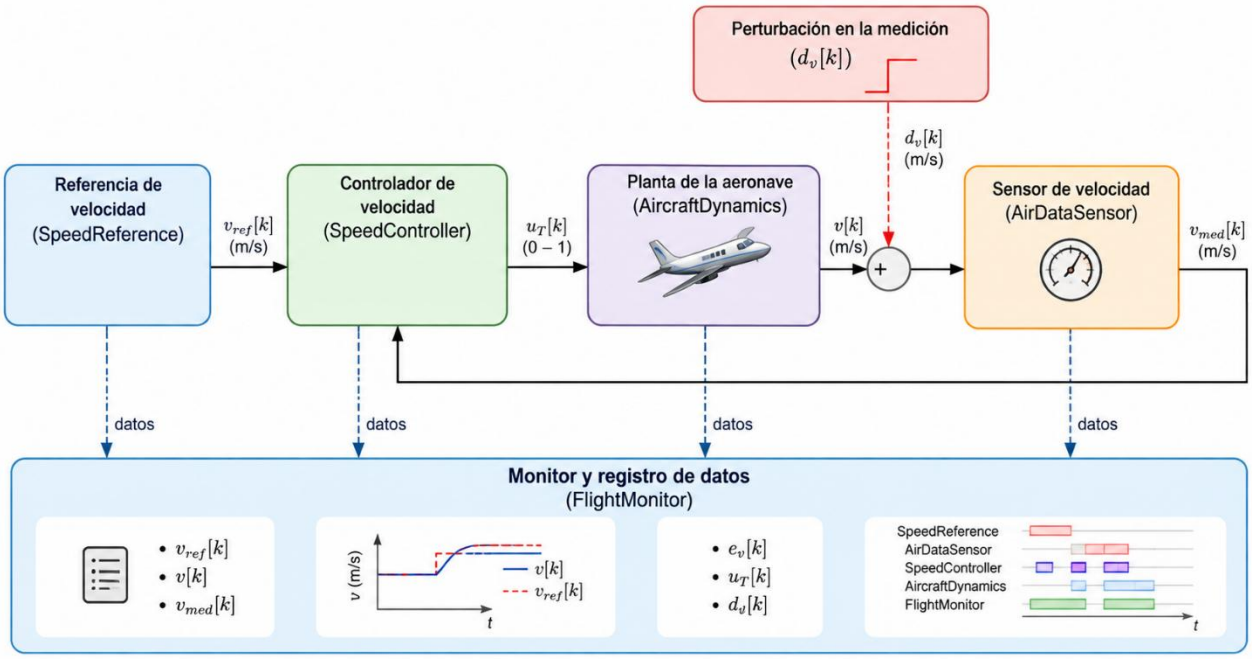


Figura 1: Diagrama de Bloques del sistema de control aéreo.

2.1. Modelo matemático sugerido:

Para todo el sistema de control indicado en la **Figura 1** se propone un modelo discreto de primer orden. El índice ***k*** identifica el instante de tiempo de actualización de la planta.

2.1.1. Medición y error

$$v_{med}[k] = v[k] + d_v[k]$$

$$e_v[k] = v_{ref}[k] - v_{med}[k]$$

2.1.2. Control proporcional de empuje

$$u_T[k] = sat\big(K_p \cdot e_v[k], 0, 1\big)$$

- Donde la saturación (sat) se define como $sat(x, 0, 1) = min\big(1, max(0, x)\big)$.
- $\boldsymbol{K_p}$ es la ganancia proporcional y debe elegirse de manera que las unidades sean coherentes y el comando no permanezca saturado innecesariamente.

2.1.3. Dinámica longitudinal

$$v[k+1] = v[k] + T_s \cdot (a_T \cdot u_T[k] - a_D \cdot v[k])$$

- Donde T_s es el período que representa el intervalo de actualización de la planta en segundos.
- a_T representa la eficacia simplificada del empuje.
- y a_D representa la resistencia aerodinámica simplificada.

Considerando que estos coeficientes son parámetros didácticos y no valores aeronáuticos, indique sus valores utilizados y justifique su selección.

2.1.4. Perturbación y cambio de referencia

- Ejemplo de perturbación en la medición:
- $d_v[k] = -5m/s$, si $10 \leq k < 20$; en otro caso, $d_v[k] = 0$
    - Esta perturbación simula un sesgo temporal del sensor de velocidad durante diez muestras medidas. Por tanto, no representa una modificación real en la velocidad física de la aeronave.
- Un segundo experimento puede ser cambiar la velocidad de referencia, v_{ref}, por ejemplo, de $100m/s$ a $110m/s$, y observar la evolución del error y del comando de empuje.

Observación general:
Este modelo es una aproximación educativa. No incluye la dinámica completa de una aeronave, cambios de densidad, ángulo de ataque, sustentación, maniobras, ni modelos detallados del motor del sistema.

2.2. Módulos y períodos requeridos:

La **Tabla 1** presenta los módulos específicos, y sus períodos requeridos, para la implementación del sistema de control de la **Figura 1**. Estos periodos son requisitos didácticos inspirados en una arquitectura *multi-rate*. Con ello, en su informe de laboratorio **explique** cómo es el tratamiento de los datos en este caso que los módulos del sistema presentan diferentes frecuencias.

Reactor / módulo	Responsabilidad	Período	Frecuencia
SpeedReference	Publica la referencia de velocidad, y permite un cambio programado de la velocidad de referencia.	100 ms	10 Hz
AirDataSensor	Realiza la medición de velocidad y aplica la perturbación simulada.	5 ms	200 Hz
SpeedController	Calcula el error y el comando de empuje saturado.	10 ms	100 Hz
AircraftDynamics	Actualiza la velocidad longitudinal simulada a partir del empuje y la resistencia.	10 ms	100 Hz
FlightMonitor	Registra referencia, medición, error, comando y marcas temporales de ejecución de cada módulo.	100 ms	10 Hz

Tabla 1: Módulos del sistema de control aéreo.

2.3. Diseño reactivo requerido:

2.3.1. El sistema debe incluir los cinco módulos (**reactors**) descritos en la **Tabla 1.** Se pueden añadir módulos auxiliares si se justifica su necesidad.

2.3.2. Defina **timers** con los períodos indicados, y conecte los puertos para que la referencia y las mediciones lleguen al controlador, el comando llegue a la planta y los datos relevantes lleguen al monitor.

2.3.3. Reacciones (**reactions**) mínimas:

i. **AirDataSensor:** Implemente al menos dos **reactions** diferentes, por ejemplo:

a. **reaction***(sensor_timer):* calcula y publica la medición de velocidad a partir de la velocidad simulada y de la perturbación vigente.

b. **reaction***(perturbation_input):* actualiza el estado de la perturbación cuando recibe un evento explícito. Si se elige otra estrategia, documente claramente qué activa esta **reaction**.

ii. **SpeedController:** Implemente al menos dos **reactions** diferentes, por ejemplo:
    a. **reaction***(controller_timer):* ejecuta el cálculo periódico del controlador y publica el comando de empuje.
    b. **reaction***(reference):* registra o valida una nueva referencia de velocidad. Si la reacción de referencia también calcula el comando, **explique** cómo evita confundir las activaciones por evento con la frecuencia periódica de control.

iii. Se recomienda que **AircraftDynamics** también se actualice mediante su **timer** y que reciba el último comando de empuje disponible.

2.4. Requisitos de implementación:

2.4.1. Defina el **lenguaje C** como el **target** final para su compilación del código **LF**, y utilice la sintaxis de **Lingua Franca** correspondiente a la versión instalada.

2.4.2. Cree un archivo principal, por ejemplo **AircraftController.lf**, con un **main reactor** que instancie los cinco módulos y conecte sus puertos.

2.4.3. Use **timers** periódicos y documente el **offset** inicial y el **período** de cada **timer**.

2.4.4. Utilice puertos tipados y conexiones **LF** para la comunicación; no los reemplace por variables globales compartidas.

2.4.5. Evite declaraciones de **scheduler**, prioridades fijas ni mecanismos manuales de planificación.

2.4.6. Distinga el período lógico de activación, el intervalo físico observado y el tiempo de ejecución de cada **reaction**.

2.4.7. Presente su código **LF** suficientemente documentado, indicando las unidades, parámetros, saturación del empuje y forma de inyección de la perturbación.

2.4.8. Asegúrese que las ecuaciones y la actualización de estado sean numéricamente estables para los parámetros elegidos.

3. Compilación y ejecución

Con la herramienta de **LF** instalada y configurada en el **PATH** de su sistema operativo, a continuación se presenta la secuencia típica de compilación y ejecución de un proyecto cuyos códigos fuente se encuentren en **src/** es:

lfc src/AircraftController.lf

bin/AircraftController

La ruta del ejecutable puede variar según el sistema operativo, el **target** escogido y la estructura del proyecto.

3.1. Diagrama generado por LF:

3.1.1. Si desarrolló más de un archivo **.lf** para la presente práctica, abra cada uno de estos archivos en su entorno de desarrollo (**vscode**, por ejemplo) y capture el diagrama de composición generado por las herramientas de **LF**.

3.1.2. En total, debe ser posible identificar los cinco **reactors** requeridos, sus puertos y conexiones. Evite presentar un dibujo manual o generado por IA, como si fuera una imagen generada por **LF**.

3.2. Evidencia de periodicidad:

3.2.1. Registre para cada activación de cada módulo del sistema, el nombre del **reactor/reaction**, la **tag lógica** y una **marca de tiempo física** de diagnóstico.

3.2.2. Presente al menos 20 activaciones por módulo indicando el período esperado, intervalo físico observado y error respecto al período.

3.2.3. Calcule el período observado medio, mínimo y máximo, desviación estándar y *jitter* (diferencia entre el intervalo observado y el período nominal).

3.2.4. Explique diferencias entre tiempo lógico y físico. Como alternativa sugerida, puede habilitar la opción de *tracing* de **LF** para el **target C**, documentada en el siguiente enlace, y adjuntar una visualización o exportación procesada.

https://www.lf-lang.org/docs/next/reference/tracing/

Tenga en cuenta que una ejecución observada no demuestra por sí sola una garantía de tiempo-real ni el cumplimiento de *deadlines* para todas las condiciones requeridas.

3.3. Evidencia del control:

3.3.1. Grafique $\boldsymbol{v_{ref}}$ y $\boldsymbol{v}$ en función del tiempo.

3.3.2. Grafique el error $\boldsymbol{e_v}$ y el comando de empuje $\boldsymbol{u_T}$, incluyendo los límites de saturación.

3.3.3. Indique en sus gráficas el instante de un cambio de referencia y/o de la perturbación del sensor.

3.3.4. Incluya una tabla de valores antes, durante y después de la perturbación.

4. Análisis y preguntas

4.1. ¿Qué restricciones de orden de ejecución provienen de las conexiones causales? ¿Qué **reactions** podrían ejecutarse concurrentemente y bajo qué condiciones?

4.2. ¿Cómo se verifica cuantitativamente que un módulo se activa cada 5, 10 ó 100 ms? Muestre los cálculos a partir de los datos obtenidos.

4.3. ¿Qué ocurre si el sensor publica mediciones más rápido de lo que el controlador calcula comandos? ¿Cómo el sistema decide qué muestra utilizar?

4.4. ¿Cómo cambia la respuesta al aumentar o disminuir $\boldsymbol{K_p}$? Presente resultados y explique la relación entre ganancia, saturación y estabilidad observada.

4.5. ¿Qué efecto tiene una perturbación en la medición frente a un cambio real de la velocidad de la planta? ¿Cómo distinguirlos en las gráficas?

4.6. ¿Qué evidencia permite afirmar que el controlador responde a la perturbación? ¿Qué evidencia adicional sería necesaria para evaluar *deadlines*?

5. Informe

5.1. Presente en su informe las respuestas a las preguntas realizadas en la presente guía de laboratorio.

5.2. Igualmente, agregue las diferentes capturas gráficas pedidas, así como su explicación del código fuente implementación, y resultados obtenidos.

5.3. Adjunte a este informe un archivo zip con el contenido de su carpeta de trabajo, albergando códigos fuente, *Makefile* y pantallazos de las salidas obtenidas en cada ejecución.

5.4. Si usó IA en la presente práctica de laboratorio, agregue una declaración formal[2] del uso de IA (Vea **aquí un ejemplo** de esta declaración).

5.5. Presente sus conclusiones generales del trabajo realizado en toda esta Práctica de Laboratorio, e indique la bibliografía utilizada para realizar este informe.

[2] **Declaración de uso de IA:** https://revistas.ucasal.edu.ar/index.php/CI/ia